

Acessar

arineto1.github.io/ifpe2026

Informática Teórica

Aula 5

Aula	Tema Principal	Foco da Teoria
1	Fundamentos e Alfabetos	Conceitos de linguagens formais, símbolos e cadeias.
2	Autômatos Finitos (AFD)	Definição formal e diagramas de estados.
3	Não-Determinismo (AFN)	Transições múltiplas e equivalência entre AFN e AFD.
4	Expressões Regulares	Relação entre álgebra de linguagens e máquinas.
5	Máquina de Turing (MT)	O modelo de fita e controle de estados universal.
6	Computabilidade	Funções recursivas e a Tese de Church-Turing.
7	Decidibilidade	Problemas decidíveis vs. indecidíveis e o Problema da Parada.
8	Análise de Algoritmos	Introdução à eficiência e Notação Big-O.
9	Complexidade Computacional	Classes de problemas P e NP.
10	Hierarquia de Chomsky	Classificação final das linguagens e revisão geral.

Alan Turing: O Pai da Era Digital

Uma jornada ilustrada pela
mente que inventou o
mundo moderno.

Digital
Revolution
Origins

Breaking the
Enigma

The Imitation
Game

From Gear to
Gigabyte: The
Legacy Lives On

The Universal
Machine Concept

Breaking
the Enigma

1930s Blueprint -
The Genesis of
Computing



O Homem, O Herói, O Gênio

O Matemático (1936)

Ferramenta do Visionário



O "Rascunho Universal"

Aos 24 anos, ele rascunhou a teoria que deu origem a todos os computadores.

Conceito Inicial: Máquina de Turing

O Herói Oculto (1940s)

Bletchley Park:
O Coração do Segredo



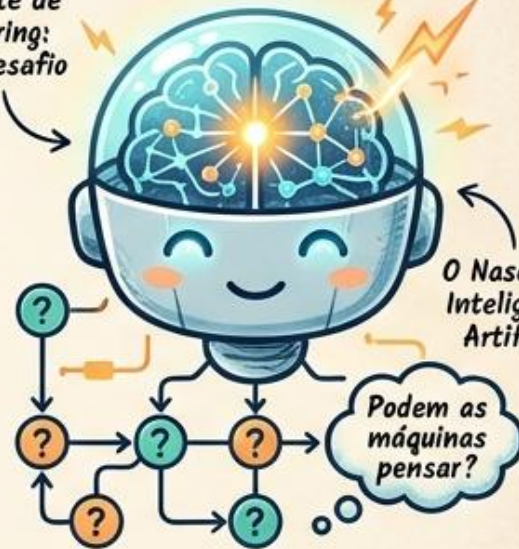
O 'Crack' Decisivo

Quebrou os códigos nazistas e encurtou a Segunda Guerra Mundial a partir das sombras.

Operação Ultrassegreta: Colossus

O Pioneiro da IA (1950s)

Teste de Turing:
O Desafio



O Nascer da Inteligência Artificial

Foi o primeiro a perguntar se as máquinas poderiam, um dia, pensar como nós.

O Legado Futuro: IA e Computação Cognitiva

1936: A Semente do Futuro.

Aos 24 anos, Turing publicou o artigo que mudaria o mundo.

Antes da invenção dos transistores ou microchips, o conceito de "computador" nasceu inteiramente na imaginação de um jovem matemático, escrito em uma simples folha de papel.



Curiosidade: O Batismo da Máquina.



Nota de Turing

O nome original: "A-Machine" (Máquina Automática). Turing era modesto e não usou seu próprio nome no artigo.



Nota de Church

A mudança: "Máquina de Turing".



Alonzo Church, seu orientador em Princeton, cunhou o termo. Church percebeu que o modelo de Turing era a descrição mais intuitiva e robusta do que significa "computar" — superando até mesmo o seu próprio modelo (o Cálculo Lambda).

O Conceito de Universalidade (Ou: O Nascimento do Software).

Hardware Fixo
A Máquina



Software
A Fita de Instruções

Software
A Fita de Instruções

*O hardware é fixo.
A função muda
através da
programação.*

Turing provou algo revolucionário: não precisamos de uma máquina diferente para cada tarefa. Uma única máquina (a **Máquina de Turing Universal**) pode simular qualquer outra máquina se receber as instruções corretas em sua fita.

O Herói Secreto de Bletchley Park

Durante a Segunda Guerra Mundial, Turing levou suas teorias matemáticas para o campo de batalha. No setor ultrassecreto "Hut 8" da inteligência britânica, ele liderou a equipe com uma missão impossível: decifrar as mensagens da Marinha Alemã.



A máquina Enigma: considerava-se que seus códigos criptografados eram inquebráveis.



Fogo contra Fogo: A 'Bombe'.

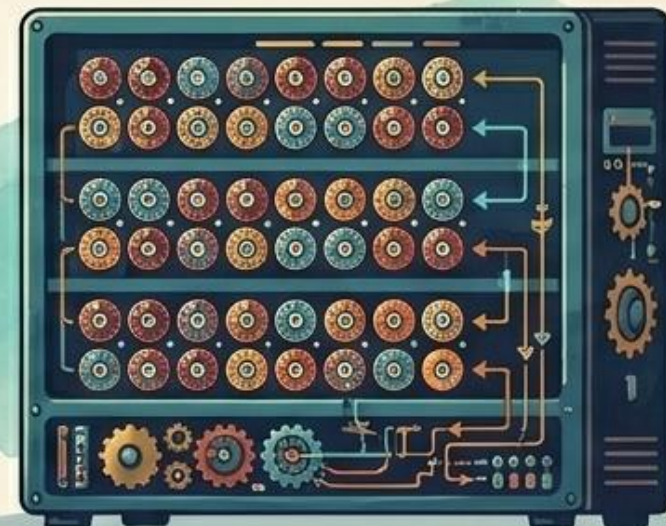
O Problema



Bilhões de combinações diárias.
Impossível para a mente humana.



A Solução



A "**Bombe**" — um gigante eletromecânico
capaz de testar milhares de configurações
em tempo recorde.

Turing percebeu que **apenas uma máquina poderia derrotar outra máquina**. Ele projetou a 'Bombe', um gigante eletromecânico capaz de testar milhares de configurações da Enigma em tempo recorde.

0 Impacto Oculto

2 Anos a Menos
de guerra

14.000.000
de vidas salvas

O trabalho de Turing e sua equipe no Hut 8 permaneceu em segredo absoluto por décadas, mas os historiadores estimam que a quebra da Enigma mudou dramaticamente o curso da história humana.

Olhando para o Futuro: O Nascimento da IA.

Após a guerra, Turing continuou desbravando fronteiras. Em 1950, ele publicou outro artigo revolucionário que deixou de focar em cálculos para fazer uma pergunta profundamente filosófica:

“**As máquinas
podem pensar?**”

*A Origem do
Teste de Turing*



*Pai da Ciência
da Computação
e da IA*

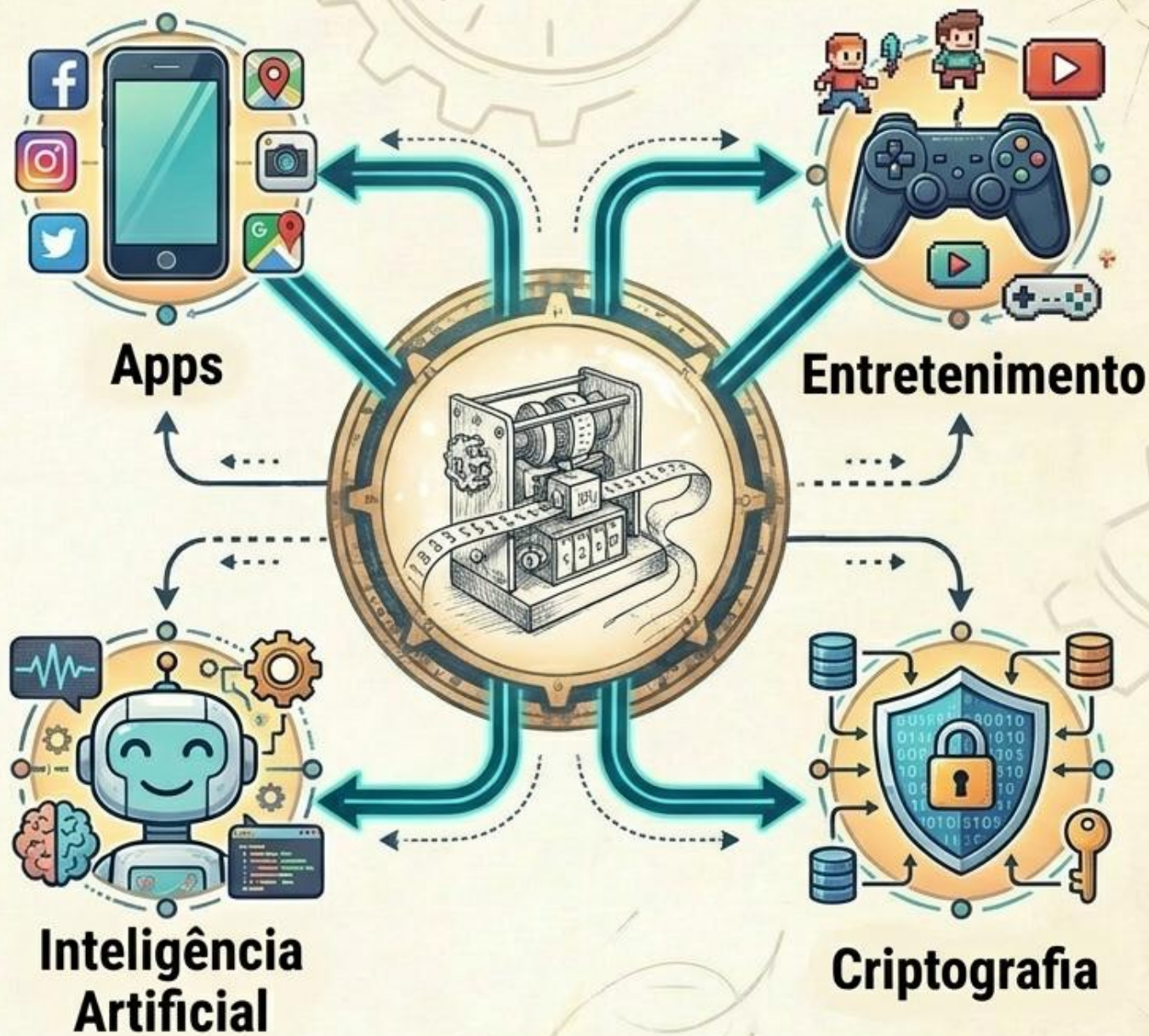
O Teste de Turing.



Ele argumentou que não deveríamos tentar definir a 'consciência'. Em vez disso, propôs um teste prático: se uma máquina consegue conversar com um humano de forma tão natural que o humano não consegue distingui-la de uma pessoa real, então devemos considerar que a máquina 'pensa'. Este é, até hoje, um pilar da IA moderna.

A Síntese: Tudo é Computação.

Em resumo, a Máquina de Turing foi o primeiro modelo a provar que a computação não depende de engrenagens ou fios específicos. É uma lei lógica universal. Todo o ecossistema digital que usamos hoje roda sob o princípio matemático que ele conseguiu isolar e descrever em 1936.



Definição de Máquina de Turing (MT)

Definição

A Máquina de Turing é um modelo matemático de computação que define uma máquina abstrata capaz de manipular símbolos em uma fita de acordo com uma tabela de regras.

Embora simples, ela é capaz de simular a lógica de qualquer algoritmo de computador, servindo como a base para o que entendemos hoje por computabilidade.

Por que ela é "Abstrata"?

Dizemos que ela é abstrata porque ela é um **modelo matemático**, não um objeto físico.

- Sem Limites Físicos
- Essência da Operação
- Universalidade

Por que ela pode simular a lógica de qualquer algoritmo?

Qualquer **algoritmo** pode ser quebrado em **três operações básicas** que a MT realiza:

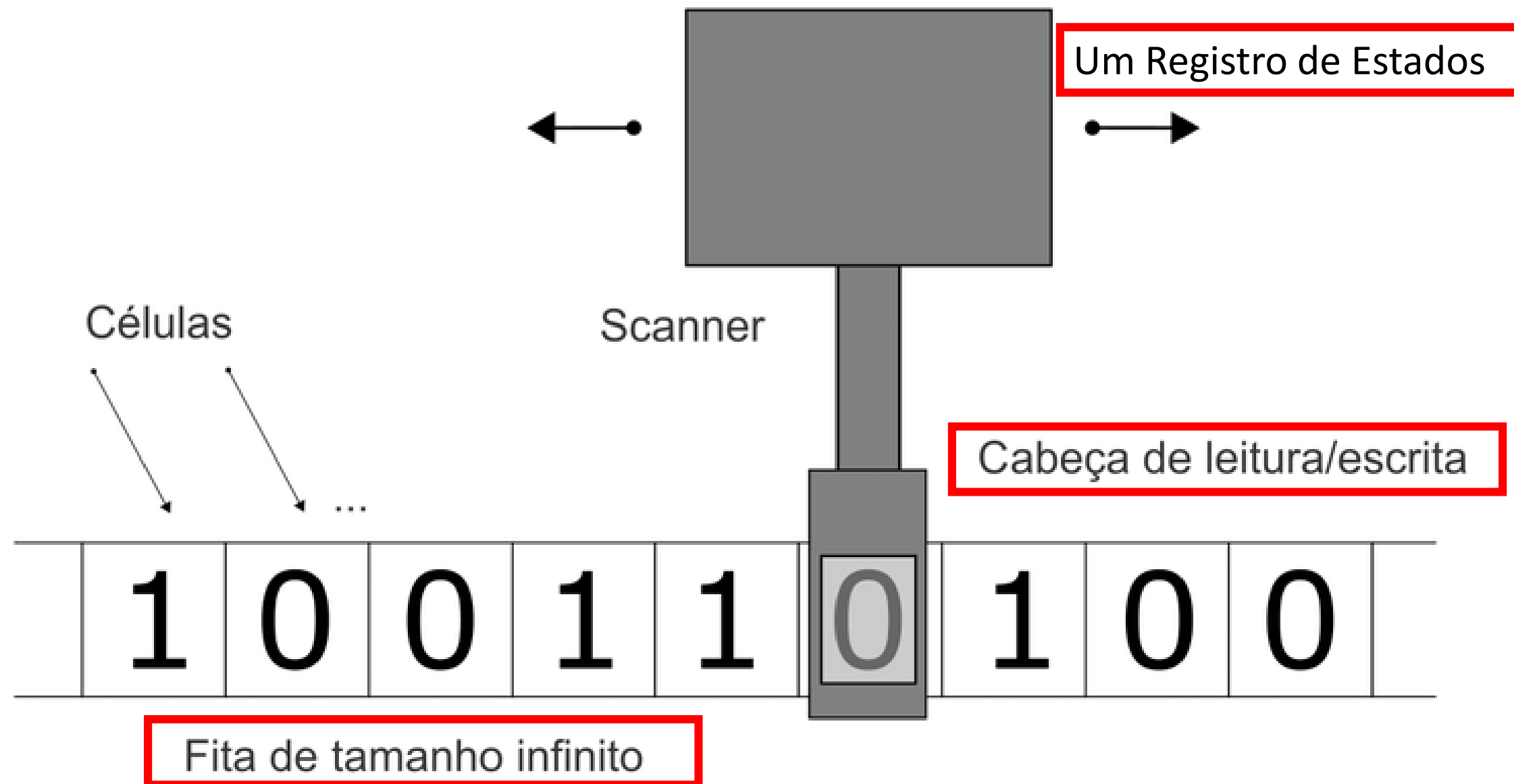
- 1.Ler/Escrever:** Mudar o estado de uma unidade de informação.
- 2.Mover:** Acessar a próxima unidade de informação.
- 3.Decidir:** Mudar o comportamento com base no que foi lido (o "if/else" primordial).

Se algo pode ser calculado por um **supercomputador da NASA**, pode ser calculado por uma **Máquina de Turing** (dado tempo e fita suficientes).

O que é Computabilidade?

A **computabilidade** é o estudo do que **pode** e do que **não pode** ser resolvido por um algoritmo.

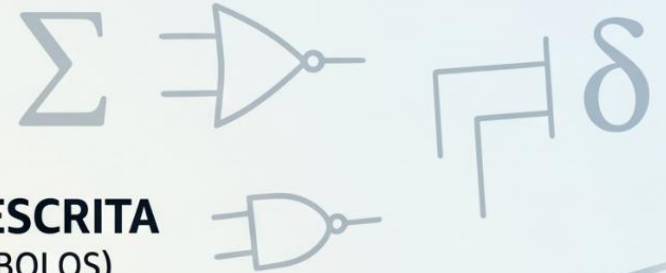
- **Problemas Computáveis:** São aqueles que a máquina consegue terminar e dar uma resposta (ex: somar dois números, ordenar uma lista).
- **Problemas Não-Computáveis:** Existem perguntas lógicas que uma máquina simplesmente entraria em um loop infinito e nunca conseguiria responder.



Algorithm notation

Algorithm:
state $\leftarrow m = \{1\}$
if ($n > 0$)
else = 0
stax (= {1})

MÁGUNA DE TURING



REGISTRO DE ESTADOS
(ARMAZENA O ESTADO LÓGICO)

CABEÇA DE LEITURA/ESCRITA
(MOVE-SE E MANIPULA SÍMBOLOS)

ESTADO ATUAL:
Q1 (SOMA)

DIREITA

ESQUERDA

FITA INFINITA
(DIVIDIDA EM CÉLULAS DE MEMÓRIA)

CÉLULA
(ENDEREÇO DE MEMÓRIA)

SÍMBOLO LIDO: 0

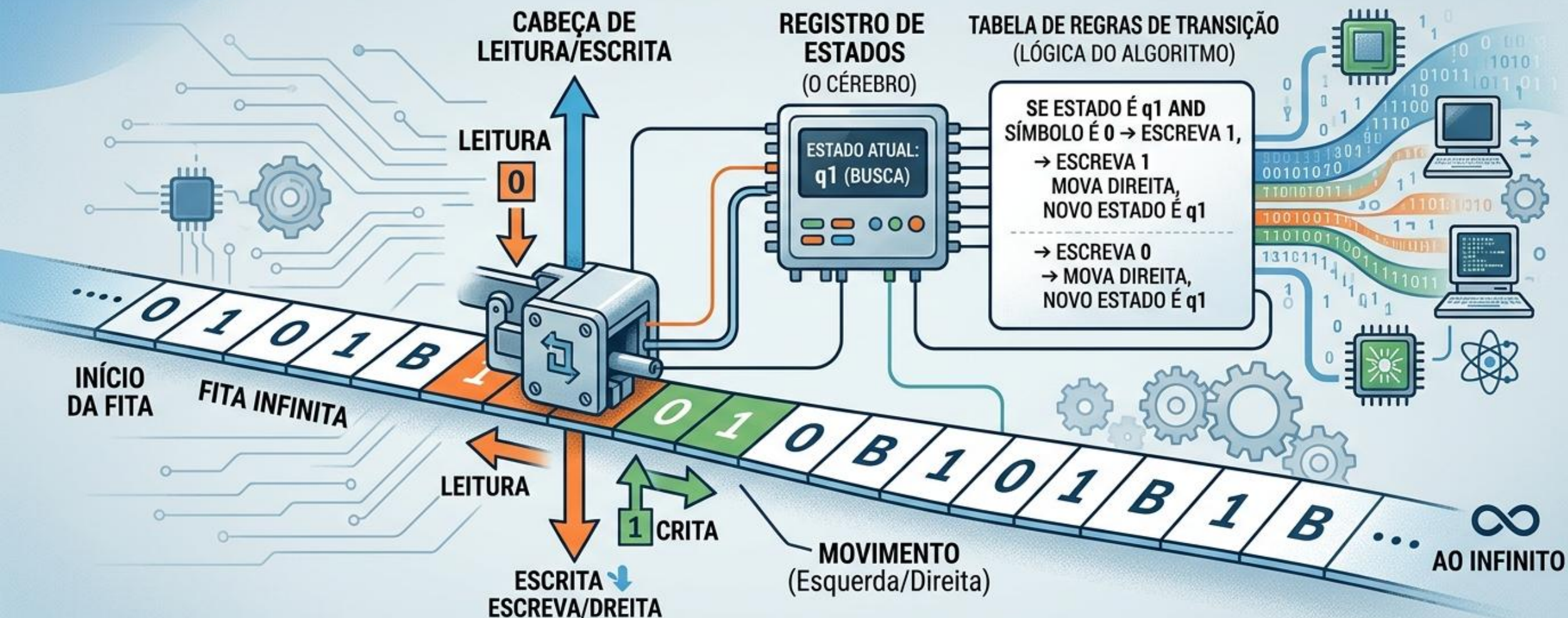
SÍMBOLOS NA FITA
(Ex: 0, 1, B)

ALGORITHMS



{1011 101010
10101 }

MÁQUINA DE TURING: O MODELO ABSTRATO DE COMPUTAÇÃO



ABSTRATA: Fita Infinita, Lógica Pura
(Sem Limites Físicos)

SIMULAÇÃO: Executa Passos Básicos
de Qualquer Algoritmo

COMPUTABILIDADE: O Que Pode e Não
Pode Ser Resolvido (Problema da Parada)

A Fita Infinita (Memória Principal)

A fita é o local onde os dados residem.

As Células: Cada quadrado na fita pode conter um único símbolo (geralmente 0, 1 ou um símbolo de "branco").

O Conceito de Infinito: Na prática, nenhuma fita é infinita, mas para a matemática, essa "infinitude" é crucial.

Entrada e Saída: A fita começa com os dados do problema (entrada) e, ao final do processo, o que sobrar escrito nela é o resultado (saída).

A Cabeça de Leitura/Escrita (O "Ponteiro")

Se a fita é o papel, a **cabeça é a caneta**. Ela é a única **interface entre o "cérebro" da máquina e a memória**.

- **Ações Limitadas:** Ela só consegue fazer três coisas por vez:
 - **Ler** o que está na célula atual
 - **Apagar e escrever** um novo símbolo
 - **Mover uma casa** para a esquerda ou para a direita.
- **Acesso Sequencial:** Para ler o milésimo símbolo, você precisa passar pelos 999 anteriores.

O Registro de Estados (O "Estado Mental")

Este é o componente mais abstrato e importante. Ele define o que a máquina está "pensando" ou qual passo do algoritmo ela está executando.

O Contexto: A máquina não tem memória de longo prazo sobre o que fez dez passos atrás. Ela só sabe em que **Estado** está agora.

Ex:

- "Estado de Busca"(**buscar símbolo**)
- "Estado de Soma"(**altera**)
- "Estado Final"(**resposta**)

Exemplo clássico: uma máquina que precisa **transformar o primeiro símbolo 0 que encontrar em um X** e depois parar.

O Cenário Inicial

- **Estado Atual:** q_0 (Início)
- **Fita:** $[0][1][1][\sqcup]$ (onde $\sqcup$ é o símbolo de espaço em branco)
- **Posição da Cabeça:** Sobre o 0 .

A Regra de Transição (A "Lógica")

Vamos definir a seguinte instrução para a máquina:

Se estiver no estado q_0 **e** ler o símbolo $\emptyset$:

1. Escreva X no lugar do $\emptyset$.
2. Mude para o estado q_{fim} .
3. Mova a cabeça para a **Direita** (R).

Formalmente, escrevemos: $\delta(q_0, \emptyset) = (q_{fim}, X, R)$

Passo 1: Leitura e Comparação

A máquina olha para a célula atual. Ela vê um 0 . Ela consulta sua tabela de regras e encontra a instrução $\delta(q_0, 0)$.

q_0 ↓
[0][1][1][␣]

Passo 2: Escrita (Modificação da Memória)

Seguindo a regra, a máquina apaga o $\emptyset$ e escreve um X .

q_0



$[X][1][1][\sqcup]$

Passo 3: Movimento e Mudança de Estado

A cabeça se desloca uma posição para a direita e o controle interno muda para o estado q_{fim} .

- Estado Atual: q_{fim}
- Fita: [X][1][1][]
- Posição da Cabeça: Agora está sobre o 1.

q_{fim} ↓
[X][1][1][]

Instante

Estado

Fita

T=0

q_0

[0][1][1]

T=1

q_{fim}

[X][1][1]

E se a máquina tivesse que encontrar o primeiro '0', mas ele estivesse lá no final da fita?

[1][1][1][0][␣]

A Lógica do Algoritmo

Para encontrar o primeiro $\emptyset$ ao final da fita, a máquina terá dois estados:

1. q_{busca} : O "scanner". Ele lê 1 , não altera nada e continua andando para a direita.
2. q_{fim} : O estado de parada após encontrar e marcar o $\emptyset$.

- **Regra de Busca:** $\delta(q_{busca}, 1) = (q_{busca}, 1, R)$
 - *Tradução:* "Se ler 1, escreva 1 (não mude nada), continue no estado de busca e mova para a direita (R)."

qbusca ↓

[1][1][1][0][␣]

- **Regra de Busca:** $\delta(q_{busca}, 1) = (q_{busca}, 1, R)$
 - *Tradução:* "Se ler 1, escreva 1 (não mude nada), continue no estado de busca e mova para a direita (R)."

qbusca ↓

[1][1][1][0][␣]

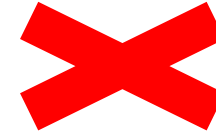
- **Regra de Busca:** $\delta(q_{busca}, 1) = (q_{busca}, 1, R)$
 - *Tradução:* "Se ler 1, escreva 1 (não mude nada), continue no estado de busca e mova para a direita (R)."

qbusca ↓
[1][1][1][0][␣]

- **Regra de Busca:** $\delta(q_{busca}, 1) = (q_{busca}, 1, R)$
 - *Tradução:* "Se ler 1, escreva 1 (não mude nada), continue no estado de busca e mova para a direita (R)."

qbusca ↓
[1][1][1][0][␣]

- **Regra de Busca:** $\delta(q_{busca}, 1) = (q_{busca}, 1, R)$



- *Tradução:* "Se ler 1, escreva 1 (não mude nada), continue no estado de busca e mova para a direita (R)."

qbusca ↓

[1][1][1][0][␣]

- **Regra de Encontro:** $\delta(q_{busca}, 0) = (q_{fim}, X, R)$
 - Tradução: "Se ler 0, escreva X, mude para o estado final e mova para a direita (R).")"

qfim ↓

[1][1][1][X][␣]

Instante	Estado	Fita
T=0	q_{busca}	[1][1][1][0]
T=1	q_{busca}	[1][1][1][0]
T=2	q_{busca}	[1][1][1][0]
T=3	q_{busca}	[1][1][1][0]
T=4	q_{fim}	[1][1][1][X]

Os 7 Componentes da Máquina Sétupla M

- **Q (Conjunto de Estados):** Um conjunto finito de estados onde a máquina pode estar. Assim como nos AFDs, isso representa a "memória de curto prazo" ou o contexto atual da execução.

Sigma

- **Σ (Alfabeto de Entrada):** O conjunto de símbolos que podem aparecer na fita inicialmente como a entrada do problema. **Importante:** O símbolo branco ($\sqcup$) nunca faz parte de Σ .

Gama

- **Γ (Alfabeto da Fita):** O conjunto total de símbolos que podem existir na fita. Ele obrigatoriamente contém todos os símbolos de Σ e o símbolo branco ($\sqcup$). É aqui que ficam os símbolos "auxiliares" (como o x que usamos no exemplo anterior).

- δ (**Função de Transição**): O "cérebro" da máquina. Sua assinatura é:

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

Ela recebe um par (Estado Atual, Símbolo Lido) e devolve um trio (Novo Estado, Símbolo Escrito, Direção do Movimento).

$$\delta(q_{busca}, 1) = (q_{busca}, 1, R)$$

$$\delta(q_{busca}, 0) = (q_{fim}, X, R)$$

- $q_0 \in Q$ (**Estado Inicial**): Onde a máquina começa sua operação.
- $q_{accept} \in Q$ (**Estado de Aceitação**): Se a máquina entrar neste estado, ela para imediatamente e a entrada é considerada **aceita**.
- $q_{reject} \in Q$ (**Estado de Rejeição**): Se a máquina entrar neste estado, ela para imediatamente e a entrada é considerada **rejeitada**.

Exercício de Fixação: O Inversor de Bits

1. Enunciado

Projete uma **Máquina de Turing (MT)** que receba em sua fita uma cadeia binária composta por símbolos do alfabeto $\{0, 1\}$. A máquina deve percorrer a fita, inverter cada bit (transformando 0 em 1 e 1 em 0) e encerrar a execução (aceitar) assim que encontrar o final da cadeia (símbolo branco $\sqcup$).

2. A Sétupla Formal

A máquina M é definida matematicamente pelos seguintes componentes:

- $Q = \{q_0, q_{aceita}\}$: Conjunto finito de estados.
- $\Sigma = \{0, 1\}$: Alfabeto de entrada.
- $\Gamma = \{0, 1, \sqcup\}$: Alfabeto da fita (inclui a entrada e o símbolo branco).
- q_0 : Estado inicial.
- q_{aceita} : Estado de aceitação.
- $q_{rejeita} = \emptyset$: Neste problema, não há condições explícitas de erro.
- δ : Função de transição (detalhada abaixo).

3. Definições Formais das Transições (δ)

A função δ define o comportamento exato da Unidade de Controle para cada par de (Estado, Símbolo). A regra segue o formato:

$$\delta(q_{atual}, leitura) = (q_{proximo}, escrita, direção)$$

1. Processamento do bit 0:

$$\delta(q_0, 0) = (q_0, 1, R)$$

(Se ler 0, escreve 1, permanece em q_0 e move para a direita).

$$\delta(q_{atual}, leitura) = (q_{proximo}, escrita, direção)$$

2. Processamento do bit 1:

$$\delta(q_0, 1) = (q_0, 0, R)$$

(Se ler 1, escreve 0, permanece em q_0 e move para a direita).

3. Condição de Parada (Fim da Cadeia):

$$\delta(q_0, \sqcup) = (q_{aceita}, \sqcup, L)$$

(Ao encontrar o vazio, mantém o vazio, transiciona para o estado final e move para a esquerda).

Passo a Passo
(Rastreamento da Fita)

[0][1][1][L]

$$\delta(q_0, 0) = (q_0, 1, R)$$

q_0 ↓

[0][1][1][␣]

q_0 ↓

[1][1][1][␣]

$$\delta(q_0, 1) = (q_0, 0, R)$$

q_0 ↓
[1][1][1][␣]

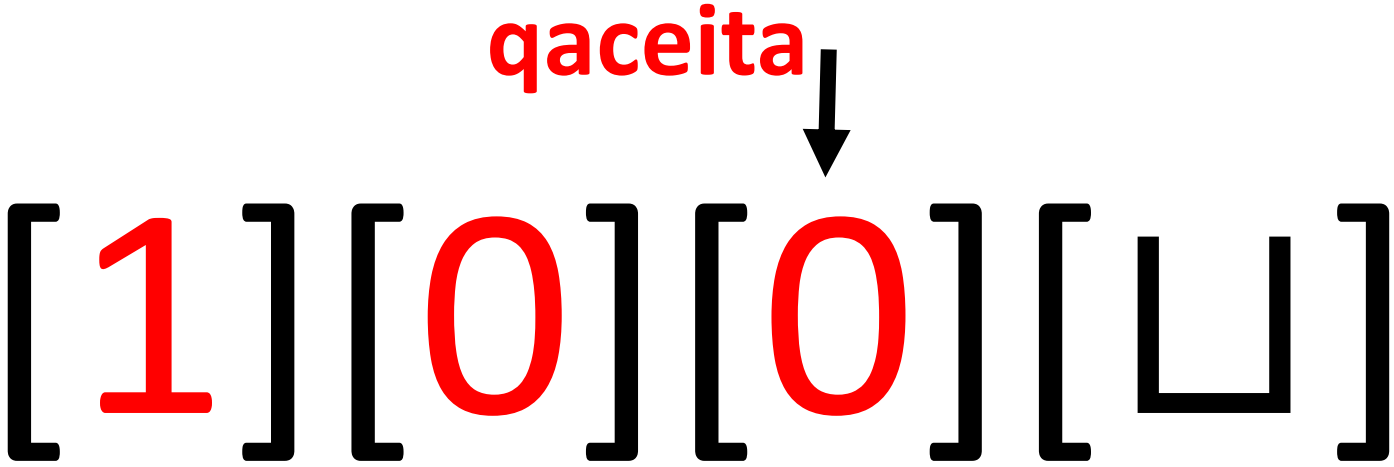
q_0 ↓
[1][0][1][␣]

$$\delta(q_0, 1) = (q_0, 0, R)$$

$q_0 \downarrow$
[1][0][1][␣]

$q_0 \downarrow$
[1][0][0][␣]

$\delta(q_0, \sqcup) = (q_{aceita}, \sqcup, L)$



Passo	Estado	Conteúdo da Fita
1	q_0	0 1 1 □
2	q_0	1 1 1 □
3	q_0	1 0 1 □
4	q_0	1 0 0 □
Fim	q_{aceita}	1 0 0 □

A Diferença Fundamental: AFD/AFN vs. MT

Diferença Fundamental: AFD/AFN vs. MT

A grande "virada de chave" para os alunos é entender que os Autômatos Finitos (AFD e AFN) são extremamente limitados em termos de memória e movimento.

Recurso	Autômatos Finitos (AFD/AFN)	Máquina de Turing (MT)
Memória	Finita e Imutável: Reside apenas nos estados.	Infinita e Editável: Possui uma fita de leitura e escrita.
Movimento	Unidirecional: Apenas para a direita (consome a entrada).	Bidirecional: Pode ir e voltar na fita livremente (L e R).
Ação	Apenas Leitura: Não pode alterar a entrada.	Leitura e Escrita: Pode modificar o conteúdo da fita.

O Que é a Tese de Church-Turing?

A Proposição Central

"Qualquer função que possa ser calculada por um algoritmo pode ser calculada por uma Máquina de Turing."

Em outras palavras, se existe um passo a passo lógico para resolver um problema, existe uma Máquina de Turing capaz de executá-lo.

Aplicação pratica da máquina de Turing

1. A Invenção do "Software" (A Máquina Universal)

Antes de Alan Turing, as máquinas eram construídas para fazer uma única coisa (como uma calculadora que só soma). Turing provou que era possível criar uma **Máquina de Turing Universal (UTM)**: uma máquina que lê a "descrição" de outra máquina na fita e a simula.

- **Onde vemos hoje:** Em tudo. Essa é a base do conceito de **computador de propósito geral**. O seu hardware não muda, mas ele pode ser um editor de texto, um jogo ou um navegador, dependendo do "manual de instruções" (software) que você carrega na memória.

2. Arquitetura de Computadores (Von Neumann)

A estrutura que usamos hoje em quase todos os dispositivos (CPU, Memória e Barramento) é uma implementação física dos conceitos de Turing.

- **Fita Infinita → RAM e SSD:** É o local onde os dados e as instruções são armazenados.
- **Cabeça de Leitura/Escrita → CPU:** O processador que busca a instrução na memória, executa a lógica e grava o resultado de volta.
- **Registro de Estados → Unidade de Controle:** Define qual instrução o processador deve executar em seguida.

3. Linguagens de Programação (Turing-Completeness)

Na ciência da computação, dizemos que uma linguagem é "**Turing-Completa**" se ela consegue simular qualquer Máquina de Turing. Isso define o poder de uma linguagem.

- **Onde vemos hoje:** Se você usa Python, JavaScript, C++ ou Java, você está usando ferramentas que têm exatamente o mesmo poder lógico que o modelo de Turing. Se algo pode ser resolvido logicamente, essas linguagens conseguem expressar isso.
- **Curiosidade:** Até ferramentas inesperadas como o **Excel**, o jogo **Minecraft** (com Redstone) e o **PowerPoint** são Turing-Completos. Isso significa que, em teoria, você poderia programar um sistema operacional inteiro dentro do Minecraft.

Resumo da Presença Atual

Conceito de Turing

Equivalente Moderno

Fita

Memória RAM / Disco Rígido

Tabela de Regras

Código Fonte / Algoritmo

Máquina Universal

O Sistema Operacional (Windows, Linux, iOS)

Mudança de Estado

Ciclo de Clock do Processador

Lista de Exercícios: Praticando Máquinas de Turing

- Criar funções de transição
- Rastreamento (Trace) da fita
- Criar diagramas de estados.

3. Definições Formais das Transições (δ)

A função δ define o comportamento exato da Unidade de Controle para cada par de (Estado, Símbolo). A regra segue o formato:

$$\delta(q_{atual}, leitura) = (q_{proximo}, escrita, direção)$$

Exercício 1: O Caçador de "1"s

Enunciado: Projete uma MT que percorra a fita para localizar o primeiro símbolo 1. Ao encontrá-lo, deve substituí-lo por um X e parar em q_{aceita} . Se encontrar um branco ($\sqcup$) antes do 1, deve parar em $q_{rejeita}$.

- **Fita de Teste:** $[0][0][1][0][\sqcup]$
- **Resultado Esperado:** A fita deve terminar como $[0][0][X][0][\sqcup]$ com a máquina em q_{aceita} .

Exercício 2: Limpando a Memória

Enunciado: Crie uma MT que apague toda a entrada da fita. Ela deve substituir cada 0 ou 1 pelo símbolo branco ($\sqcup$) e só aceitar quando encontrar o final da cadeia original.

- **Fita de Teste:** $[1][0][1][\sqcup]$
- **Resultado Esperado:** A fita deve terminar completamente limpa $[\sqcup][\sqcup][\sqcup][\sqcup]$ com a máquina em q_{aceita} .

Exercício 3: O Marcador de Fim

Enunciado: Desenvolva uma MT que caminhe até o final de uma cadeia binária e, no primeiro espaço disponível (branco), escreva o símbolo #. Após escrever, a cabeça deve se mover uma posição para a esquerda e aceitar.

- **Fita de Teste:** $[0][1][1][\sqcup]$
- **Resultado Esperado:** A fita deve terminar como $[0][1][1][\#]$ com a máquina em q_{aceita} .

Exercício 4: Padronizador de Bits (Tudo "1")

Enunciado: Projete uma MT que transforme qualquer sequência em uma cadeia de apenas 1s. Todo 0 deve ser reescrito como 1. Se o símbolo já for 1, ele deve ser mantido. A máquina aceita ao chegar no fim da cadeia.

- **Fita de Teste:** [1][0][0][1][$\sqcup$]
- **Resultado Esperado:** A fita deve terminar como [1][1][1][1][$\sqcup$] com a máquina em q_{aceita} .

Exercício 5: O Filtro de Início

Enunciado: Crie uma MT que verifique apenas o primeiro caractere da fita.

1. Se for $\emptyset$, a máquina escreve A (de Aceito), move para a direita e entra em q_{aceita} .
 2. Se for qualquer outra coisa (incluindo branco), ela entra em $q_{rejeita}$.
- **Fita de Teste A:** $[\emptyset][1][1][\sqcup]$ (Deve aceitar e resultar em $[A][1][1][\sqcup]$)
 - **Fita de Teste B:** $[1][1][\emptyset][\sqcup]$ (Deve rejeitar imediatamente)